33

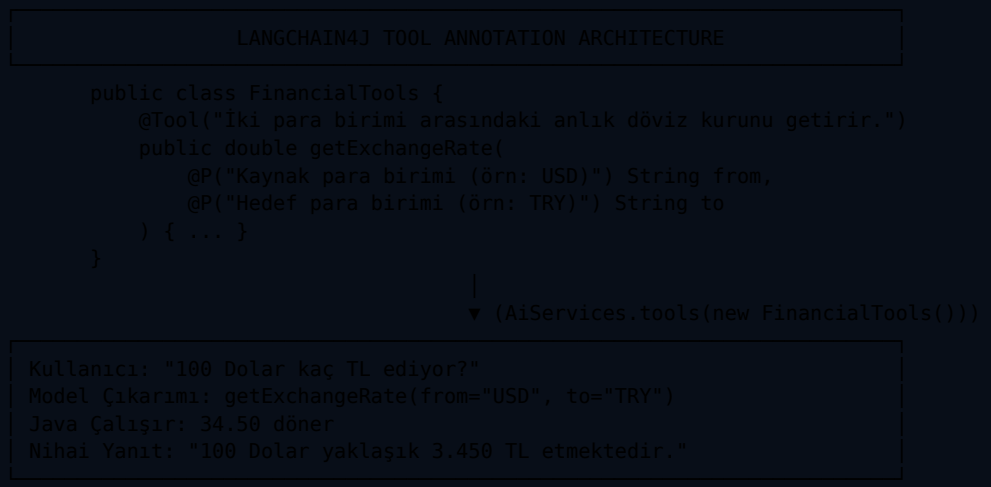
BÖLÜM: FUNCTION CALLING, TOOL USE VE API ENTEGRASYONU | MODULE: FUNCTION CALLING, TOOL INTEGRATION & AGENT TOOLING PROTOCOLS

LangChain4j @Tool Açıklamaları ve Dinamik Parametre Çıkarımı

LangChain4j Tool Engineering: Parameter Extraction & Strict Typings

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Akıllı bir hesap makineniz olduğunu düşünün. Normalde '100 Dolar kaç TL eder' dediğinizde ekranda hata verir. Ama bu hesap makinesinin arkasına bir döviz bürosu telefonu bağlarsanız (@Tool), hesap makinesi 'Dur bir dakika, önce kuru öğreneyim' der, döviz bürosunu arar, kuru alır ve size doğru sonucu hesaplar! LangChain4j'in `\@Tool` ve `\@P` etiketleri bu telefon numaralarını ve konuşma kurallarını tanımlar!

🇬🇧 ENGLISH (ELI5)

Imagine a standard pocket calculator. If you type 'Convert 100 USD to Euros', it displays an error syntax. But if you connect the calculator to a live currency trading terminal via a smart cable (@Tool), the calculator pauses, queries the live exchange rate, performs the multiplication, and returns the result. LangChain4j's `\@Tool` and `\@P` annotations build these live conduits between models and your internal microservices!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

LangChain4j Tool Parametre Çıkarımı: 1) `@Tool`: Metodun LLM'e açılmasını sağlar. 2) `@P`: Parametre düzeyinde açıklama (`description`) sağlar. 3) Tip Güvenliği (Strong Typing): Model parametreleri String, int, double, enum ve hatta Java Record nesneleri olarak üretebilir; LangChain4j JSON string'ini ilgili Java tipine otomatik cast eder. 4) Çoklu Tool Yönetimi: Bir `@AiService`'e onlarca tool bağlanabilir; model bir istekte hangi tool'u veya birden fazla tool'u sırayla çağıracağını kendisi planlar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

LangChain4j Tool Resolution Engine: 1) `@Tool` Annotation: Marks methods for reflection inspection and JSON Schema generation. 2) `@P` Parameter Documentation: Injects descriptions into property schemas. 3) Coercion and Deserialization: Automatically casts JSON primitives to target Java types (including complex nested records and enums). 4) Multi-Tool Execution: AiServices supports injecting diverse tool classes simultaneously; the model autonomously chains multiple distinct tool calls to resolve compound tasks.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & LangChain4j: @Tool ile Finansal Hesaplama Ajanı
import dev.langchain4j.agent.tool.P;
import dev.langchain4j.agent.tool.Tool;
import dev.langchain4j.service.AiServices;
import dev.langchain4j.model.openai.OpenAiChatModel;

public class FinancialAgentService {
    public static class ExchangeTools {
        @Tool("Verilen iki para birimi arasındaki canlı piyasa kurunu sorgular.")
        public double getRate(
            @P("Kaynak para birimi, örn: USD, EUR") String from,
            @P("Hedef para birimi, örn: TRY") String to
        ) {
            if ("USD".equalsIgnoreCase(from) && "TRY".equalsIgnoreCase(to)) return 34.50;
            return 1.0;
        }
    }

    public interface CurrencyAssistant {
        String answer(String userQuery);
    }

    void main() {
        var model = OpenAiChatModel.builder()
            .apiKey(System.getenv("OPENAI_API_KEY"))
            .modelName("gpt-4o-mini")
            .temperature(0.0)
            .build();

        CurrencyAssistant assistant = AiServices.builder(CurrencyAssistant.class)
            .chatLanguageModel(model)
            .tools(new ExchangeTools()) // Tool enjeksiyonu!
            .build();

        IO.println("LangChain4j Tool Ajanı Başlatıldı.");
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Hata Yönetimi: Eğer Tool çalıştığı sırada veritabanı çökerse veya bir exception fırlatılırsa, tüm AI uygulaması patlamamalıdır! Tool metodları try-catch ile korunmalı ve hata durumunda modele 'Kargo servisine şu an ulaşılamıyor' şeklinde anlamlı bir metin dönmelidir.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Exception handling standard: Tool methods must never leak unhandled runtime exceptions. If a database or HTTP service fails, catch the error and return a descriptive message ('Database unavailable, please try again') so the LLM can explain the failure gracefully.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- LangChain4j \@Tool ve \@P açıklamaları, tip güvenli parametre çıkarımı sağlar.
- Enum ve Record tipleri, modelin parametre halüsinasyonu görmesini engeller.
- Tool içi hatalar try-catch ile yakalanıp modele doğal dilde hata mesajı olarak iletilmelidir.

🎯 Key Takeaways for the Interview

- LangChain4j \@Tool and \@P annotations guarantee strongly-typed argument extraction.
- Enums and Record types constrain models to valid domain values.
- Tool exceptions must be captured and fed back as clean error context to the model.